

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ
АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ ИТМО»

ФАКУЛЬТЕТ ТЕХНОЛОГИЙ ИСКУССТВЕННОГО ИНТЕЛЛЕКТА



Отчёт по домашней работе №5

«Разработка одностраничного веб-приложения (SPA) с
использованием фреймворка Vue.JS»

Семестр 4

Выполнил: студент
Мавров Артём Николаевич
гр. J3213
ИСУ 466574

Отчёт сдан: 26.04.2026

Проверил: Добряков Д. И.

Санкт-Петербург
2026

Содержание

1	Задача	3
2	Ход работы	3
2.1	Инициализация проекта	3
2.2	Организация API-слоя	3
2.3	Настройка Pinia	4
2.4	Подключение зависимостей в main.js	5
2.5	Компонентная архитектура	5
2.6	Роутинг и navigation guard	6
2.7	Работа с v-model, v-for, v-if, mounted	6
3	Вывод	7

Задача

В рамках данной домашней работы необходимо изучить основные команды пакетного менеджера NPM и научиться стартовать проект на Vue.js. Требуется перенести ранее реализованный сайт с чистого HTML/CSS/JS на фреймворк Vue.js, следуя структуре и подходам, описанным в прилагаемом мануале: инициализация проекта через `npm init vue@latest`, настройка зависимостей (`axios`, `pinia`, `bootstrap`), организация API-слоя, хранилищ состояния и компонентной архитектуры.

Ход работы

Инициализация проекта

Проект инициализирован командой `npm init vue@latest` с подключением Vue Router и Pinia:

```
1 npm install -g create-vue
2 create-vue mlpipe-vue --router --pinia
3 cd mlpipe-vue
4 npm install
```

Листинг 1: Инициализация Vue-проекта

После инициализации в файле `package.json` добавлен скрипт `start` для запуска на порту 8080, как указано в мануале:

```
1 "scripts": {
2   "start": "vite --port 8080",
3   "dev": "vite",
4   "build": "vite build"
5 }
```

Листинг 2: Скрипт запуска в package.json

Затем установлены необходимые зависимости:

```
1 npm install axios pinia-plugin-persistedstate bootstrap -S
```

Листинг 3: Установка зависимостей

Организация API-слоя

По аналогии с мануалом создана директория `src/api/`. Общий инстанс `axios` вынесен в `src/api/instance.js`. В отличие от мануала, инстанс дополнен интерцептором запросов, который автоматически подставляет токен авторизации из `localStorage`:

```
1 import axios from 'axios'
2
3 const instance = axios.create({
4   baseURL: 'http://localhost:3001',
5   headers: { 'Content-Type': 'application/json' }
6 })
7
8 instance.interceptors.request.use((config) => {
9   const token = localStorage.getItem('mlpipe_token')
```

```

10   if (token)
11       config.headers['Authorization'] = 'Bearer ' + token
12   return config
13 })
14
15 export default instance

```

Листинг 4: Файл `src/api/instance.js`

Для каждого ресурса создан отдельный API-класс по паттерну из мануала (DI через конструктор). Файл `src/api/index.js` является IoC-контейнером:

```

1 import instance      from '@api/instance'
2 import AuthApi       from '@api/auth'
3 import ExperimentsApi from '@api/experiments'
4 import ModelsApi     from '@api/models'
5 import ArtifactsApi  from '@api/artifacts'
6
7 const authApi        = new AuthApi(instance)
8 const experimentsApi = new ExperimentsApi(instance)
9 const modelsApi      = new ModelsApi(instance)
10 const artifactsApi   = new ArtifactsApi(instance)
11
12 export { authApi, experimentsApi, modelsApi, artifactsApi }

```

Листинг 5: IoC-контейнер `src/api/index.js`

Настройка Pinia

Файл `src/stores/index.js` создан по образцу из мануала. Вместо `pinia-persists` использован актуальный пакет `pinia-plugin-persistedstate`:

```

1 import { createPinia } from 'pinia'
2 import piniaPluginPersistedstate
3   from 'pinia-plugin-persistedstate'
4
5 const pinia = createPinia()
6 pinia.use(piniaPluginPersistedstate)
7
8 export default pinia

```

Листинг 6: Настройка Pinia с плагином persist

Для каждого ресурса создано отдельное хранилище: `auth.js`, `experiments.js`, `models.js`, `artifacts.js`. Каждое хранилище содержит `state`, `getters` и `actions` по паттерну из мануала:

```

1 export const useExperimentsStore = defineStore('experiments', {
2   state: () => ({
3     experiments: []
4   }),
5   actions: {
6     async loadExperiments() {
7       const res = await experimentsApi.getAll()
8       this.experiments = res.data
9       return res

```

```

10     },
11     async createExperiment(data) {
12         await experimentsApi.create(data)
13         await this.loadExperiments()
14     }
15 }
16 })

```

Листинг 7: Фрагмент хранилища experiments.js

Подключение зависимостей в main.js

Файл `src/main.js` переписан по образцу из мануала с подключением `pinia`, `router` и стилей `Bootstrap`:

```

1 import { createApp } from 'vue'
2 import App from '@/App.vue'
3 import router from '@/router'
4 import pinia from '@/stores'
5 import 'bootstrap/dist/css/bootstrap.min.css'
6 import 'bootstrap'
7 import '@/assets/main.css'
8
9 const app = createApp(App)
10 app.use(pinia)
11 app.use(router)
12 app.mount('#app')

```

Листинг 8: Файл `src/main.js`

Компонентная архитектура

Проект разделён на три уровня согласно мануалу: представления (`src/views/`), компоненты (`src/components/`) и лейауты (`src/layouts/`).

Реализованы следующие представления:

Файл	Назначение
<code>LoginPage.vue</code>	Страница входа
<code>RegisterPage.vue</code>	Страница регистрации
<code>DashboardPage.vue</code>	Дашборд с KPI-счётчиками
<code>ExperimentsPage.vue</code>	Список экспериментов с фильтрами
<code>ExperimentDetailPage.vue</code>	Детальная страница эксперимента
<code>ModelsPage.vue</code>	Реестр моделей
<code>ArtifactsPage.vue</code>	Артефакт-стор
<code>ProfilePage.vue</code>	Профиль пользователя

Лейаут `AppShell.vue` выполняет роль обёртки, объединяя компоненты `AppSidebar.vue` и `AppTopbar.vue` со слотом для контента. Использование `<slot>` полностью соответствует подходу из мануала:

```

1 <template>
2   <AppShell title="                ">

```

```

3     <template #actions>
4         <button class="btn-sm" @click="showCreate = true">
5
6         </button>
7     </template>
8     <!-- -->
9 </AppShell>
10 </template>

```

Листинг 9: Использование AppShell в представлении

Роутинг и navigation guard

Маршруты настроены в `src/router/index.js` с ленивой загрузкой представлений. Добавлен navigation guard, который перенаправляет неавторизованных пользователей на страницу входа:

```

1 router.beforeEach((to) => {
2     const auth = useAuthStore()
3     if (to.meta.requiresAuth && !auth.isLoggedIn) {
4         return { path: '/login' }
5     }
6     if ((to.path === '/login' ||
7         to.path === '/register') && auth.isLoggedIn) {
8         return { path: '/dashboard' }
9     }
10 })

```

Листинг 10: Navigation guard в router/index.js

Работа с v-model, v-for, v-if, mounted

В представлениях активно применяются директивы Vue из мануала. Пример формы создания эксперимента с `v-model` и отправкой:

```

1 <div v-if="createError" class="err-msg">
2     {{ createError }}
3 </div>
4
5 <input v-model="newExp.name"
6     class="finput"
7     placeholder="my-experiment" />
8
9 <button class="btn-accent"
10     @click="doCreate"
11     :disabled="creating">
12     {{ creating ? '...' : '...' }}
13 </button>

```

Листинг 11: Форма с v-model и условным рендерингом

Хук `mounted` используется во всех представлениях для загрузки данных с сервера при открытии страницы:

```

1  mounted() {
2      this.loadExperiments()
3  }
4  \end{lstlisting}
5
6  \subsection{                                }
7
8
9
10                                     :
11
12 \begin{lstlisting}[language=bash,
13     caption={                                }]
14 #                                1                                hw5 (json-server)
15 npm install json-server@0.17.4
16 node Server.js                    # http://localhost:3001
17
18 #                                2                                hw5/mlpipe-vue
19 npm install
20 npm start                          # http://localhost:8080

```

Листинг 12: Хук `mounted` для загрузки данных

При занятом порту 8080 Vite автоматически выбирает следующий доступный порт (8081), о чём сообщает в консоли.

Вывод

В ходе домашней работы сайт полностью перенесён с чистого HTML/CSS/JS на фреймворк Vue.js в соответствии с мануалом.

Освоены следующие инструменты и подходы: инициализация проекта через `npm init vue@latest`; установка и подключение зависимостей через `npm` (`axios`, `pinia`, `bootstrap`); организация API-слоя через классы с `dependency injection` и IoC-контейнер; управление состоянием через `Pinia` с персистентностью в `localStorage`; компонентная архитектура с разделением на представления, компоненты и лейауты; клиентский роутинг через `Vue Router` с ленивой загрузкой и `navigation guard`; директивы `v-model`, `v-for`, `v-if` и хук `mounted`.

Проект успешно собирается командой `vite build` (164 модуля без ошибок) и запускается на `http://localhost:8080`.